

Об'єктно-орієнтоване програмування

Частина 1: класи, екземпляри, атрибути, методи

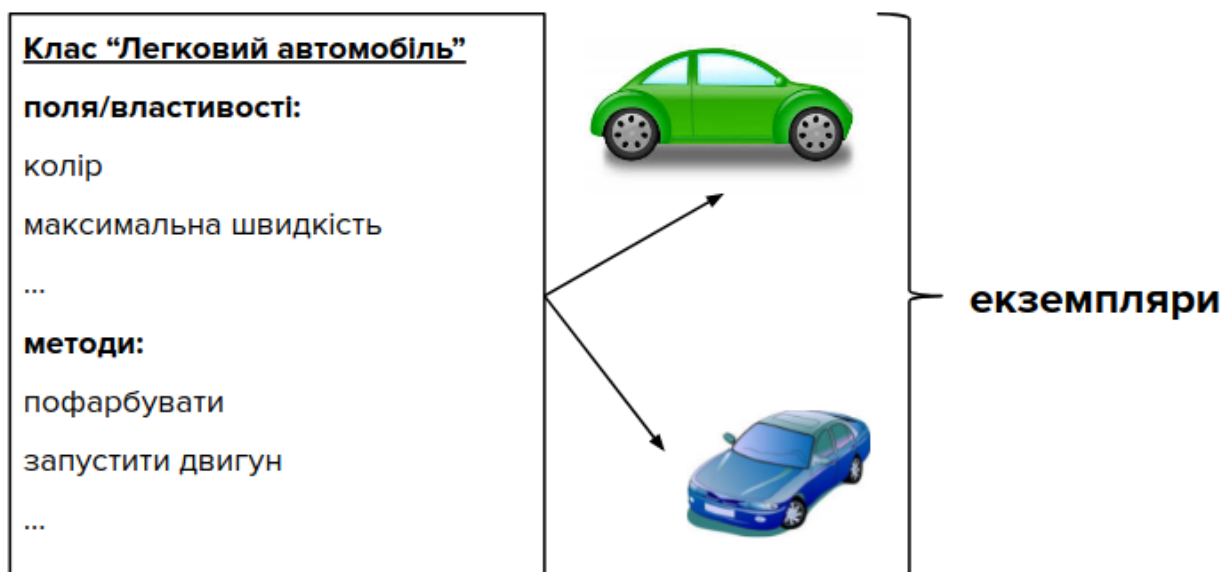
У Python усе є об'єктами. Але попри те, що наші програми містили об'єкти — використовували їх у виразах, передавали їх функціям, викликали їхні методи та ін. — до цього часу вони не писалися в об'єктно-орієнтованому стилі.

Об'єктно-орієнтоване програмування (надалі — ООП) виникло внаслідок розвитку парадигми процедурного програмування, в межах якої об'єкти і підпрограми (процедури та функції) формально не пов'язані. ООП розглядає програму як множину об'єктів, що взаємодіють між собою.

Як і у процедурному програмуванні, в ООП програма розкладається на складові. Але якщо ці складові використовуються у нових програмах, вони зазвичай не видозмінюються, а адаптуються.

В основному ООП використовується, коли програмні продукти передбачають довгострокову розробку і супровід.

В ООП розрізняють два різновиди об'єктів — класи і екземпляри. Перші реалізують поведінку і можливість створення других, а другі — власне об'єкти, які обробляються програмою.



Класи та екземпляри

Клас — це програмна компонента для реалізації нових типів об'єктів, яка характеризується станом і поведінкою та підтримує наслідування. Класи є відображенням об'єктів реального світу в програмному коді та слугують своєрідними фабриками об'єктів.

Кожного разу під час виклику класу створюється новий об'єкт (**екземпляр** цього класу), який, крім доступу до всіх атрибутів класу, має свій власний простір імен для своїх даних, відмінних від даних інших екземплярів цього ж класу. Цим класи чимось нагадують модулі з функціями, але, на відміну від останніх, мають свої особливості, основними з яких є підтримка створення багатьох екземплярів, наслідування та перевантаження операторів.

Класи прийнято іменувати, починаючи з великої літери, при цьому слова-складові імені теж починаються з великої літери, а символи підкреслення не використовуються (такий стиль ще називають верблюдячим, **CamelCase**).

Для створення класу використовується складена інструкція `class`. Для прикладу створимо клас `Person`, який в майбутньому репрезентуватиме особу:

```
class Person:
    pass # порожня інструкція, яка резервує місце під тіло класу
```

На відміну від класів, які створюються інструкціями, екземпляри створюються викликами. Попри те, що клас `Person` на даному етапі порожній, є можливість створювати екземпляри цього класу:

```
alice = Person()
bob = Person()
```

В реальних програмних продуктах класи часто поміщають в окремі файли (модулі), які потім імпортуються до файлу основної програми. Так, якщо опис класу `Person` знаходиться у файлі `myclasses.py`, то **інстанціація** екземпляру в файлі основної програми матиме вигляд:

```
import myclasses
alice = myclasses.Person()
```

або

```
from myclasses import Person
bob = Person()
```

Атрибути та методи

Атрибути класу описують стан екземплярів, а **методи** — притаманну їм поведінку. Атрибути (змінні, визначені всередині класу) створюються інструкціями присвоєння, а методи (функції, визначені всередині класу) — вкладеними в інструкцію `class` інструкціями `def`.

Усі атрибути та методи доступні через точкову нотацію, при цьому кожен екземпляр успадковує всі атрибути та методи класу:

```
об'єкт.атрибут    # доступ до атрибуту
об'єкт.метод()    # доступ до методу
```

Операція присвоєння створює **атрибут класу**, спільний для всіх майбутніх екземплярів цього класу.

```
class Person:
    info = "I'm a Person."
```

Щоб вказати, що повинен створитися атрибут, значення якого різні для різних екземплярів, як ім'я об'єкта використовується `self`. `self` також завжди передається першим аргументом в заголовках методів, які використовують атрибути класу.

Розширимо наведений вище клас `Person`, додавши до нього метод `create()` для задання атрибутів (імені та віку особи) відповідно:

```
class Person:
    info = "I'm a Person."

    def create(self, name, age):
        self.name = name
        self.age = age
```

Щоб метод `create()` зміг присвоїти певні значення атрибутам `name` (ім'я) і `age` (вік), ці значення мають бути передані йому через аргументи (як зазначалося раніше, першим аргументом завжди виступає `self`). Наприклад, в інструкції `self.name = name` точкова нотація `self.name`

позначає атрибут `name` екземпляра класу, а `name` — значення, яке передається аргументом методу `create()` і буде присвоєне цьому атрибуту.

Тобто, змінна `name` як локальна змінна доступна лише всередині методу `create()`, а атрибут `self.name` доступний з будь-якого місця всередині класу (це буде використано нижче у методі `__str__()`).

Продемонструємо роботу методу `create()` на прикладі створеного раніше екземпляра `alice`, який насправді до цього часу не має ані імені, ані віку (`alice` - просто ім'я змінної):

```
alice.create('Alice', 20)
```

Тепер, використовуючи точкову нотацію, можна отримати безпосередній доступ до атрибутів екземпляра:

```
print(alice.name, alice.age)      # Alice 20
```

```
alice.age += 1  
print(alice.age)                  # 21
```

Конструктор

В ООП імена методів Python-класів, які починаються і закінчуються двома символами підкреслення, мають спеціальне призначення. Одним з таких методів є метод `__init__()`, який називається **конструктором** класу і викликається автоматично в момент створення екземпляра. Метою конструктора є ініціалізація початкових значень атрибутів.

У попередньому прикладі новостворений екземпляр `alice` класу `Person` був "порожнім", він набував змісту тільки після задання конкретних значень його атрибутам викликом методу `create()`.

Зазвичай перші значення присвоюються атрибутам у конструкторі класу. Оскільки конструктор викликається автоматично, то, змінивши ім'я `create` на `__init__`, ми одним рядком зможемо створювати "повноцінні" (з початковими значеннями атрибутів) екземпляри:

```
class Person:  
    info = "I'm a Person."  
  
    def __init__(self, name, age):
```

```
self.name = name
self.age = age
```

```
alice = Person('Alice', 20)
bob = Person('Bob', 25)
```

Початкові значення значень атрибутів екземпляра передаються у круглих дужках після імені класу під час інстанціації екземпляра у тому ж порядку, у якому наведений перелік цих атрибутів в заголовку конструктора після першого обов'язкового аргумента self.

Деякі особливості атрибутів

Зазначимо, що атрибут класу info буде спільний для всіх екземплярів:

```
print(alice.info)      # I'm a Person.
print(bob.info)        # I'm a Person.
```

Якщо змінити атрибут класу через об'єкт екземпляра, то зміниться значення атрибуту тільки для цього екземпляра, а значення атрибута класу (і інших екземплярів цього класу) залишиться незмінним:

```
bob.info = "I am not a Person :)"
print(alice.info)      # I'm a Person.
print(bob.info)        # I am not a Person :)
```

Змінювати такі атрибути потрібно через об'єкт класу:

```
Person.info = "I am human."
print(alice.info)      # I am human.
print(bob.info)        # I am human.
```

Існує також можливість створення нових атрибутів екземпляру:

```
alice.gender = 'female'
```

Але у такій формі вона зазвичай не використовується, оскільки новостворені таким чином атрибути не доступні методам класу без їх ручного оновлення. Крім того, цей атрибут буде мати лише екземпляр alice:

```
print(alice.gender)    # female
print(bob.gender)
AttributeError: 'Person' object has no attribute 'gender'
```

Метод `__str__()`

При виведенні на екран значення екземпляру функцією `print()` відобразиться інформація про відповідний об'єкт та його розташування в пам'яті комп'ютера, наприклад:

```
print(alice)          # <__main__.Writing object at 0x7f1ffee53198>
```

Для виведення змістовної інформації про екземпляр класу `Person`, використовується спеціальний метод `__str__()`, який інструкцією `return` повертає рядок у потрібному для відображення форматі. Доповнимо наш клас цим методом:

```
class Person:
    info = "I'm a Person."

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}, {self.age} years old."
```

і продемонструємо його роботу:

```
print(alice)          # Alice, 20 years old
```

Тобто, метод `__str__()` призначений для рядкового представлення інформації про стан об'єкта (формує рядок зі значеннями атрибутів об'єкта у зручній для сприйняття формі), яке за замовчуванням передається аргументом функції `print()` при спробі надрукувати цей об'єкт.

Зауважте, що передавання `self` методу `__str__()` забезпечує непотрібність передавання імені та віку для їх виводу на екран, оскільки після виконання інструкцій присвоєння у конструкторі їх значення стануть атрибутами класу і будуть доступні через об'єкт `self`.